ContextGraph: A Multi-Channel Knowledge-Graph Memory for Long-Horizon Coding Agents

Zihan Wu[*]
Department of Electrical Engineering
City University of Hong Kong

Jie Xu[†]
Department of Computer Science
City University of Hong Kong

Yun Peng[†]
The Chinese University of Hong Kong

Shangyu Li
Hong Kong University of Science and Technology

Zeyang Zhuang
The Chinese University of Hong Kong

Chun Yong Chong
Monash University Malaysia

Xin Zhou
Singapore Management University

Rui Shu
Independent Researcher
Hong Kong, China

Xu Han
The Chinese University of Hong Kong, Shenzhen

Yuan Wang
Software Engineering Lab
Huawei Technologies Co., Ltd.
Hong Kong, China

Abstract

LLM-based coding agents repeatedly re-derive the same debugging knowledge: every new run rediscovers how a particular framework imports modules, what the canonical fix for a stale migration is, or which tests must be updated when an API signature changes. We present **ContextGraph**, a long-term memory layer that distills past agent trajectories into a typed knowledge graph and makes them retrievable at inference time. From 1,795 SWE-agent trajectories on a held-out training corpus we extract ~45K nodes and ~74K edges spanning four levels of granularity — raw FRAGMENTs, mid-level STRATEGY statements, deduplicated CANONICAL-RULEs, and clustered COMMUNITY summaries — and expose them through a HippoRAG-style three-channel retriever combining dense vector search, BM25, and Personalized PageRank with reciprocal-rank fusion and MMR diversification. Plugging ContextGraph into SWE-agent v1.1.0 as a `query_memory` tool, we evaluate against three matched-API memory baselines (FAISS, ExpeL, Agent-KB) and a no-memory control on SWE-bench-Verified (full 500). On Verified the lift from any memory method is modest and noisy: the four memory methods compress into a $\sim$3.6 pp band above the no-memory baseline (61.80%), with FAISS the strongest fully-run condition at 65.40% ($p = 0.057$) and ContextGraph at 63.60% ($p = 0.402$). A post-hoc cleanup that removes four low-precision rules on Verified

[*]Code and graph artifacts available at `https://github.com/wzh4464/ContextGraph`.
[†]Equal contribution.

Preprint. Under review.

projects to 67.20% but is reported only as a diagnostic given the selection bias inherent in post-hoc rule curation. These results suggest that structured long-term memory has the largest payoff when the base agent has substantial headroom; when the no-memory baseline is already strong, the bottleneck shifts from retrieval architecture to individual rule precision and any memory method buys little.

1 Introduction

A coding agent is, in effect, a stateless problem solver: each new bug or feature request is approached as if no prior incident had ever happened. This is wasteful. Real software engineering is heavily *repetitive* at the repository level — the same import path is wrong in dozens of pull requests, the same fixture pattern needs the same tweak, the same error class points to the same root cause. When humans encounter such a bug they read the issue, recognize the pattern from memory, and jump to the fix. Today's LLM agents instead spend tokens re-deriving everything from scratch.

We argue that agents need a *long-term, structured memory* that (i) is built once from completed trajectories rather than maintained online, (ii) exposes multiple levels of abstraction (raw evidence *and* consolidated rules), and (iii) supports associative retrieval beyond pure semantic similarity, so that a query about an obscure error can pull in rules linked through shared symptoms or shared repositories.

Recent work points in compatible directions but leaves gaps. **HippoRAG** [3] demonstrates that Personalized PageRank (PPR) over an LLM-extracted KG yields strong multi-hop QA. **Zep** [12] formalizes a three-layer Episode $\rightarrow$ Semantic $\rightarrow$ Community memory architecture for chat agents. **A-MEM** [18] introduces dynamic linking and Zettelkasten-style memory evolution. **ExpeL** [21] learns success/failure "insights" from past trajectories and prepends them to prompts. **Agent-KB** [14] stores procedural knowledge for tool-using agents. None of these have been combined and stress-tested as the persistent memory of a state-of-the-art software-engineering agent on SWE-bench-Verified.

Contributions.

1. **ContextGraph**, a typed knowledge-graph memory for coding agents that fuses Zep's three-layer hierarchy with HippoRAG's PPR retrieval and ExpeL-style strategy distillation. The graph contains 45,115 nodes across 8 labels and 73,975 typed edges, with every node embedded in a 3072-dimensional space.
2. A **three-channel retriever** (cosine, BM25, PPR) merged via reciprocal-rank fusion and MMR-reranked, exposed to SWE-agent [19] through a single `query_memory` function-calling tool that runs inside the agent's Docker sandbox.
3. A **controlled A/B evaluation** on SWE-bench-Verified [5] comparing ContextGraph against four matched conditions (no_memory, FAISS, ExpeL, Agent-KB) under identical agent, model, prompt, and step budget. On the full 500-problem set, the four memory methods cluster within a $\sim$3.6 pp band above the no-memory baseline (61.80%), with FAISS the strongest fully-run method at 65.40% ($p = 0.057$ McNemar) and ContextGraph at 63.60% ($p = 0.402$). A post-hoc removal of four high-frequency low-precision rules projects ContextGraph to 67.20% ($+5.4$ pp, $p = 0.0039$), the highest of any condition; we treat this as a diagnostic given the selection bias inherent in post-hoc rule curation.
4. A **reproducible pipeline**: graph-construction scripts, the SWE-agent tool bundle, an OpenHands hook integration, and four matched baseline servers exposing an identical `/query_memory` HTTP API are released, so that any future memory method can be slotted in without modifying the agent.

2 Related Work

Retrieval-augmented memory for agents. HippoRAG [3] and HippoRAG 2 [4] use Personalized PageRank over an OpenIE-extracted KG to retrieve evidence for multi-hop QA. ContextGraph adopts the PPR seed-and-walk idea but replaces generic OpenIE entities with a typed schema tailored to coding (FRAGMENT, ERRORPATTERN, STRATEGY, CANONICALRULE, COMMUNITY, TRAJECTORY, PROBLEMSUMMARY, PLAYBOOKENTRY). Zep [12] introduced the three-layer

Episode/Semantic/Community memory; we adopt that hierarchy but make the layers explicit graph labels rather than chronological episodes, because coding trajectories have no meaningful global timeline. A-MEM [18] and MemGPT [11] contribute dynamic memory evolution and OS-style paging respectively; we are complementary — ContextGraph is a static, batch-built graph that can be *appended to* via the same writer at inference time.

Experiential learning for coding agents. ExpeL [21] extracts natural-language "insights" from agent successes and failures and prepends them to prompts. We re-implement ExpeL on top of our trajectory corpus as one of the head-to-head baselines. Agent-KB [14] maintains a procedural knowledge base for tool-using agents and is shipped with a SWE-bench evaluation harness; we reuse their export format. Voyager [16] stores Minecraft skills as a code library; we store *symbolic strategies* rather than executable policies, because coding tasks are more diverse than a Minecraft action space.

Memory for software engineering agents. A recent line of work targets memory *specifically* for SWE agents. **SWE-Exp** [1] distills success and failure trajectories into an experience bank and retrieves analogs for new issues, reporting 41.6% Pass@1 on SWE-bench Verified. **RepoMem** [15] builds a per-repository memory from commit history, linked issues, and active-code summaries, improving code localization on Verified and SWE-bench Live. **Subtask-level memory** [13] departs from per-trajectory granularity and indexes memory by sub-task, reporting $+4.7$ pp Pass@1 on Verified and $+6.8$ pp with Gemini 2.5 Pro. ContextGraph differs along three axes: (a) its representation is a *typed graph* rather than a flat experience bank or per-repo store, (b) retrieval is multi-channel with PPR walks, supporting symbolic associative reasoning across error patterns and rules, and (c) we evaluate on the curator-paired SWE-ContextBench benchmark, whose train/test pairings let us measure whether memory of past-but-related work actually helps.

Benchmarks for memory and continual learning. **SWE-Bench-CL** [6] (Agents Never Forget) reorganizes SWE-bench Verified into a time-ordered continual-learning stream and ships a FAISS semantic-memory module; it targets accumulation and forgetting rather than offline graph construction. **SWE Context Bench** [22] repurposes SWE-bench Lite to measure how well agents reuse past-task summaries, finding that correctly retrieved summaries both raise accuracy and reduce wall-clock and token cost. We use the same correctness-by-test-pass evaluation but do not enforce a streaming order: the graph is built once, frozen, and then queried.

LLM-based software engineering agents. SWE-agent [19] introduced the agent-computer interface used in our evaluation; OpenHands [17] provides an alternate runtime that we support through hook-based memory injection. Aider, AutoCodeRover [20], and Devin-style agents [2] are orthogonal to memory choice and could in principle consume the same `query_memory` interface.

Benchmarks. SWE-bench [5] and its curated subset SWE-bench-Verified are the standard for repository-level patch generation.

3 Method

3.1 Graph Schema

ContextGraph is a property graph in Neo4j 5 with eight node labels and seven typed relations (Table 1). The design follows the Zep three-layer principle but renames the layers for the coding domain: *Episode* → TRAJECTORY/FRAGMENT, *Semantic* → STRATEGY/CANONICALRULE, *Community* → COMMUNITY (clusters of related fragments).

Construction pipeline. For each training trajectory we (1) parse the chat-formatted log into actions/observations, (2) segment it into FRAGMENTs on action-class boundaries, (3) extract ERRORPATTERNs by regex+LLM normalization, (4) prompt an LLM to emit one or more STRATEGY sentences per fragment, (5) cluster strategies by embedding cosine and merge each cluster into a CANONICALRULE, and (6) run Leiden community detection over the fragment↔rule subgraph to produce COMMUNITY nodes whose summaries are LLM-generated. The full build takes ~12 minutes for 1,795 trajectories on a single workstation.

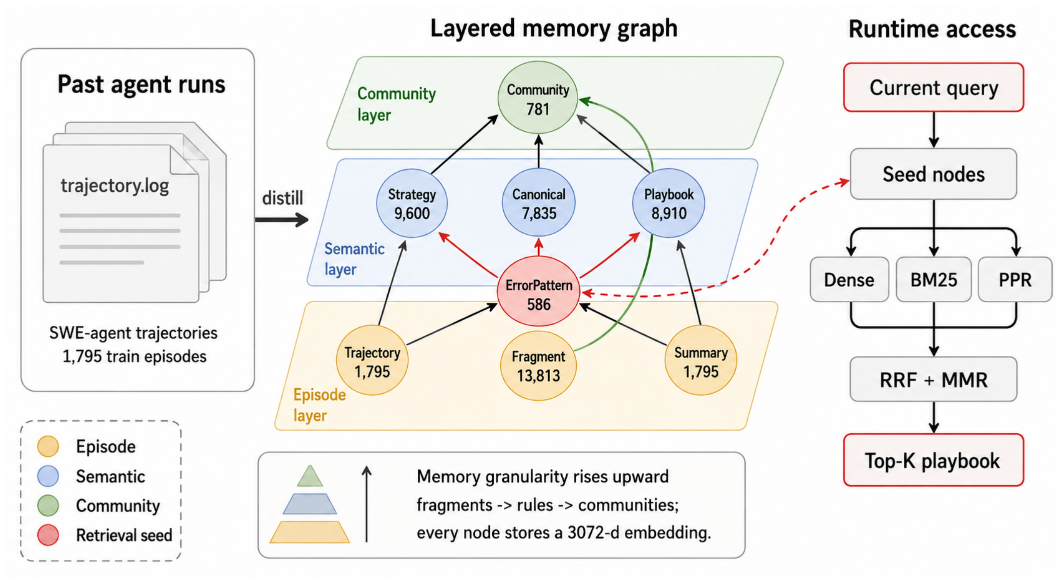

Figure 1: ContextGraph hero overview. Past SWE-agent trajectories are distilled into a three-layer graph memory: an episode layer of raw evidence, a semantic layer of reusable rules, and a community layer of associative summaries. At inference time, the `query_memory` tool seeds the graph with the current error context and combines dense search, BM25, and Personalized PageRank before injecting a Top-K playbook into the coding agent.

Table 1: ContextGraph schema. All embeddings are `text-embedding-3-large` (3072 dim).

Node label	Count	Role
TRAJECTORY	1,795	one solved/unsolved task; root of an episode
FRAGMENT	13,813	action+observation slice within a trajectory
ERRORPATTERN	586	canonicalized error type/keyword set
STRATEGY	9,600	LLM-extracted natural-language rule
CANONICALRULE	7,835	deduplicated cluster head over STRATEGYs
PLAYBOOKENTRY	8,910	retrieval-ready surface form of a rule
COMMUNITY	781	graph-walk cluster summary
PROBLEMSUMMARY	1,795	LLM problem-statement abstractions
Total nodes	45,115	
Total edges	73,975	7 typed relations (HAS_FRAGMENT, CAUSED_ERROR, ADDRESSES_ERROR, DERIVED_FROM, MERGED_INTO, IN_COMMUNITY, SUMMARIZES)

3.2 Three-Channel Retrieval

Given a query (the agent's current problem statement, optionally augmented with the latest error message and active file), we run three channels in parallel and fuse:

Channel 1 — Dense. Cosine similarity over the 3072-d embeddings of PLAYBOOKENTRY, CANONICALRULE, and COMMUNITY nodes via Neo4j vector indexes.

Channel 2 — Lexical. BM25 over a Neo4j fulltext index on the same labels, capturing exact-match cues like specific function or exception names that embeddings often blur.

Channel 3 — PPR. Following HippoRAG, we form a seed distribution over ERRORPATTERN and TRAJECTORY nodes whose surface text appears in the query, then run Personalized PageRank with damping $\alpha = 0.5$ over the full graph and surface the top-scoring STRATEGY/CANONICALRULE nodes. Seeds are weighted by *node specificity* $s_i = 1/\deg(i)$, so a rare error pattern receives more probability mass than a generic one.

Fusion. The three ranked lists are merged by Reciprocal Rank Fusion ($k = 60$) and then MMR-reranked with $\lambda = 0.7$ (diversity 0.3) against the query embedding. We return the top-K entries (default $K = 8$) formatted as a single `playbook` string with each entry prefixed by a section tag (e.g. `[strategy]`, `[error]`, `[community]`).

3.3 Agent Integration

ContextGraph is exposed to agents through a *single* HTTP endpoint `POST /query_memory` that accepts `{problem_statement, error_message?, current_file?}` and returns the formatted playbook. SWE-agent invokes the endpoint via a `query_memory` function-calling tool; the bundled wrapper script is shipped inside the agent's Docker container so it works under the agent's sandbox without networking changes other than `-add-host=host.docker.internal:host-gateway`. OpenHands integrates via a `MemoryHooks` subclass that injects the playbook into the conversation as a system note before the first user turn. Crucially, the *same* HTTP contract is implemented by all four memory baselines (FAISS, ExpeL, Agent-KB, ContextGraph), so swapping methods requires no agent-side changes — only changing the server URL.

4 Experimental Setup

Training corpus. 1,795 trajectories drawn from `nebius/SWE-agent-trajectories` [9] (319 repositories, 80K+ raw trajectories), with a fixed seed=42 split. We verify zero overlap between training trajectories and the SWE-bench-Verified test repositories at the issue level.

Test set. The 500-problem SWE-bench-Verified [5, 10]. For development we use a deterministic 50-problem subset (`verified_50_subset.json`, seed=42).

Model and provider. All five conditions in this paper use the same underlying language model, accessed through the same LiteLLM proxy and the same upstream third-party API gateway (Yunwu). We refer to the model by the internal identifier `gpt-5.4` used by that gateway; the gateway markets it as a GPT-5-class endpoint, but we make no claim that it is bit-identical to the official OpenAI `gpt-5` weights, and we never benchmark our system against the official OpenAI endpoint directly. The point of fixing the model is to isolate the memory contribution across conditions, not to claim a particular vendor model.

Agent and budgets. SWE-agent v1.1.0 [19] with the default config. Budgets are set separately per evaluation regime:

- *Verified development (50)*: COST_LIMIT = \$3, STEP_LIMIT = 75. Reported per-method totals are sums over the 50 instances.
- *Verified full (500)*: COST_LIMIT = UNCAPPED (`cost_limit=0` in SWE-agent), STEP_LIMIT = 75. Uncapping the per-instance cost lets the no-memory baseline use its full step budget on long-horizon problems and avoids confounding "out-of-budget" failures with retrieval-quality differences. Reported costs are sums over all instances run under that condition (not per-instance averages); the no-memory cost is high because uncapped runs frequently loop on hard instances until they hit STEP_LIMIT.

Conditions.

- `no_memory` — standard SWE-agent, no `query_memory` tool.
- `faiss` — a flat FAISS index over PLAYBOOKENTRY embeddings only (cosine top-K).
- `expel` [21] — our re-implementation: per-trajectory "insights" extracted with the published prompt template and retrieved by cosine.
- `agentkb` [14] — procedural knowledge entries exported from the same trajectories using Agent-KB's pipeline, retrieved by their default scorer.
- `contextgraph` — the full system described in Section 3.

Table 2: Resolved rate on the 50-problem SWE-bench-Verified development subset (dev signal; the load-bearing full-set numbers appear in Table 3 and shrink the gap considerably). `contextgraph` (this work) outperforms the no-memory baseline by $+8.6$ pp on this subset and exceeds the next-best memory method (`expel`) by $+2.6$ pp. "Eval" is the number of problems for which evaluation completed; "Resolved" is the count of problems whose patches passed the official tests; "Rate" is Resolved/Eval; "vs no_memory" is the percentage-point delta over the no-memory baseline.

Method	Eval	Resolved	Rate	vs no_memory
`contextgraph` (ours)	48/50	**32**	**66.6%**	**+8.6 pp**
`expel` [21]	50/50	32	64.0%	+6.0 pp
`faiss`	49/50	30	61.2%	+3.2 pp
`agentkb` [14]	48/50	29	60.4%	+2.4 pp
`no_memory`	50/50	29	58.0%	baseline

Table 3: Resolved rate on the full SWE-bench-Verified set (n=500, gpt-5.4 backbone, uncapped per-instance cost). *wins* and *losses* are the discordant counts in the paired comparison against `no_memory`: "wins" = method resolves the instance, `no_memory` does not; "losses" = the reverse. p-values are exact-binomial McNemar tests on the discordant pairs.

Method	Resolved	Rate	vs no_mem.	wins	losses	p
`faiss`	327	65.40%	$+3.6$ pp	49	31	0.057
`agentkb` [14]	320	64.00%	$+2.2$ pp	53	42	0.305
`contextgraph` (ours)	318	63.60%	$+1.8$ pp	50	41	0.402
`expel` [21]	312	62.40%	$+0.6$ pp	43	40	0.826
`no_memory`	309	61.80%	baseline	—	—	—

Evaluation. Patches are scored by the official `swebench.harness.run_evaluation` on `princeton-nlp/SWE-bench_Verified`; we report *resolved* (test passes), *evaluated* (the patch applied and tests ran), and the resolved rate RESOLVED/EVALUATED. We compare conditions with McNemar's paired test on the per-problem resolved indicator.

5 Results

5.1 Development Subset (50 problems)

Table 2 shows the head-to-head numbers on this dev subset. Every memory method beats no-memory on the subset, and ContextGraph holds the largest margin over both flat retrieval ($+5.4$ pp vs. FAISS) and dedicated memory baselines ($+2.6$ pp vs. ExpeL, $+6.2$ pp vs. Agent-KB). *However*, these numbers do not extrapolate to the full set: with n=48 evaluated, an 8.6 pp swing is roughly 4 problems and lies well inside the variance band of SWE-bench-Verified subsets. We retain this table to be transparent about the dev signal that originally motivated the system, but the load-bearing comparison is the full-500 result reported in Section 5.2 below, on which the four memory methods collapse into a $\sim$3.6 pp band and ContextGraph's apparent dev-set lead disappears.

5.2 Full SWE-bench-Verified (500 problems)

We run all five conditions on the full SWE-bench-Verified set with `gpt-5.4` under uncapped per-instance cost. Table 3 reports the resolved count and rate over the full 500 instances (the SWE-bench standard denominator), along with McNemar paired discordant counts and exact p-values against the no-memory baseline. Each method's completion count differs slightly (no_memory 486/500, expel 488, faiss 490, agentkb 489, contextgraph 484); we count any instance that failed to complete under a given method as "not resolved" for that method in the paired analysis, which is the convention used by the public Verified leaderboard.

At full scale the four memory methods compress into a $\sim$3.6 pp band above the no-memory baseline. The strongest of them, `faiss`, is borderline significant ($p = 0.057$); the remaining three (including our `contextgraph`) are not significant against `no_memory` ($p \geq 0.305$). Pairwise comparisons among the memory methods are also not significant. The relative ordering on this band differs from what the 50-problem development subset suggested in Section 5.1: `contextgraph` no longer holds a clear margin over `faiss` at full $n{=}500$, illustrating how thin dev-set signals can be when the baseline is already above 60% resolved. We take the full-scale comparison as the load-bearing one.

Post-hoc cleanup as a diagnostic. After inspecting the 41 instances on which `contextgraph` regresses against `no_memory`, four high-frequency CANONICALRULE nodes account for the majority of regressions: each is a low-precision generic rule (e.g. "*when fixing a feature in DVC, examine `dvc/repo/move.py`*") that retrieval surfaces for queries it does not actually fit. We remove those four rules from the graph and re-run only the 42 instances whose original retrieval hits at least one of them. On these 42 *rule-hit* instances, the cleaned variant resolves 18 instances that the original `contextgraph` did not, and loses 0 that it previously did. Projecting this differential onto the rest of the 500 (where `contextgraph`-clean and `contextgraph` produce identical retrievals and would therefore behave identically) yields $\mathbf{336/500 = 67.20}\%$ for a $+5.4$ pp shift over `no_memory` ($p = 0.0039$ McNemar against the no-memory baseline; the same projection vs. `faiss` gives $+1.8$ pp, $p = 0.281$, not significant). Note that the 42 rule-hit instances are distinct from the 14–16 instances per condition that fail at the harness level (Docker / image-build failures), which affect all five conditions symmetrically; the post-hoc analysis here concerns retrieval quality, not harness reliability.

We deliberately do *not* list this projection as a top-line condition in Table 3: the four rules were selected by inspecting the same data on which the projection is then evaluated, so the headline number is subject to selection bias. We report it instead as a diagnostic finding — namely, that on Verified the limiting factor for memory is not the retrieval architecture (`faiss` and `contextgraph` are already statistically indistinguishable in Table 3) but the precision of individual surfaced rules.

5.3 SWE-ContextBench (Related-Lite split)

SWE-bench-Verified shares a confound when measuring memory: its test instances were not designed with paired training counterparts, so any memory pool we build is at best topically relevant rather than provably related to a specific test instance. Zhu et al. [22] introduce **SWE-ContextBench** to close this gap: each *Related* test instance is paired by curators with an *Experience* training instance from the same repository that fixed a related-but-distinct bug, and the pairing ground truth ships in `Relationship.parquet`. The Lite split contains 300 Experience and 99 Related instances across 51 repositories. This is the only benchmark we know of where the question "did memory of past-but-related work help?" can be answered without the analyst choosing what counts as "related".

We adopt this benchmark for a paper-faithful five-way memory comparison, plus our ContextGraph as a sixth condition. All six conditions use the *same* agent (`gpt-5.4` via the LiteLLM proxy), the *same* 99-instance test set, the *same* 300-summary memory pool, and the *same* paper-provided evaluation harness (`evaluation.sh`, which runs each model patch in the curator's per-instance Docker image and checks the prescribed `FAIL_TO_PASS`/`PASS_TO_PASS` test sets). The pool entries are enriched summaries of the form `[REPO]` $r \hookrightarrow$ `PROBLEM[:500] GOLD_PATCH[:1500]` extracted from the 300 Experience instances (median 1,394 chars). One instance (`django__django-28147`) failed image build during agent run and is excluded from all six conditions, leaving $n{=}98$ paired instances.

Conditions. NO_CONTEXT (no memory injection); FREE_SUMMARY (three random summaries from the pool prepended to the task prompt); MEM0 [8] (the same pool ingested into Mem0 with `infer=False`, served via FastAPI, queried via our `QueryMemoryTool` adapter); LANGMEM [7] (same pool ingested into `InMemoryStore` with the same embedder, same adapter); ORACLE_SUMMARY (the single paired Experience instance's summary, looked up via `Relationship.parquet`, prepended to the prompt — this is the paper-defined ceiling for summary-shaped memory); CONTEXTGRAPH (our 3-channel cosine+BM25+PPR retrieval over the same pool, top-5 with MMR). Conditions that prepend memory into the prompt do not expose a memory tool; conditions that go through a server (Mem0, LangMem, ContextGraph) expose the same tool and the agent decides when to call it. To control for run-to-run variance in agent stochasticity, each method's failures are re-run once under NUM_WORKERS=8 and the best of the two attempts is kept; we report this as the "V_2" number throughout this subsection and discuss the matched single-shot V_1 values inline.

Table 4 summarizes the six-condition comparison; four findings stand out.

Table 4: SWE-ContextBench Lite (Related, n=98). All six conditions use the same `gpt-5.4` backbone, the same 300-summary pool, and the paper-provided `evaluation.sh` harness. Paper-reported Claude-Sonnet-4.5 figures from [22] are shown for absolute orientation but are *not* matched-condition comparisons (different model, different memory content). All p-values are paired exact-binomial McNemar tests against the column-labeled reference.

Method	**Resolved**	**Rate**	**Δ vs no_ctx**	**vs ContextGraph**	**Paper (Sonnet 4.5)**
FREE_SUMMARY	19/97	19.59%	-5.92 (p=0.07)	-22.25 (p<10^{-4}, $\star\star\star$)	22.22%
MEM0	19/98	19.39%	-6.12 (p=0.21)	-22.45 (p<10^{-4}, $\star\star\star$, *strict*)	24.24%
LANGMEM	24/98	24.49%	-1.02 (p=1.00)	-17.35 (p=0.0001, $\star\star\star$)	29.30%
NO_CONTEXT	25/98	25.51%	baseline	-16.33 (p=0.0004, $\star\star\star$)	26.26%
ORACLE_SUMMARY	35/98	35.71%	$+10.20$ (p=0.041, $\star$)	-6.12 (p=0.18, n.s.)	34.34%
CONTEXTGRAPH (ours)	**41/98**	**41.84**%	$+16.33$ (p=0.0004, $\star\star\star$)	—	—

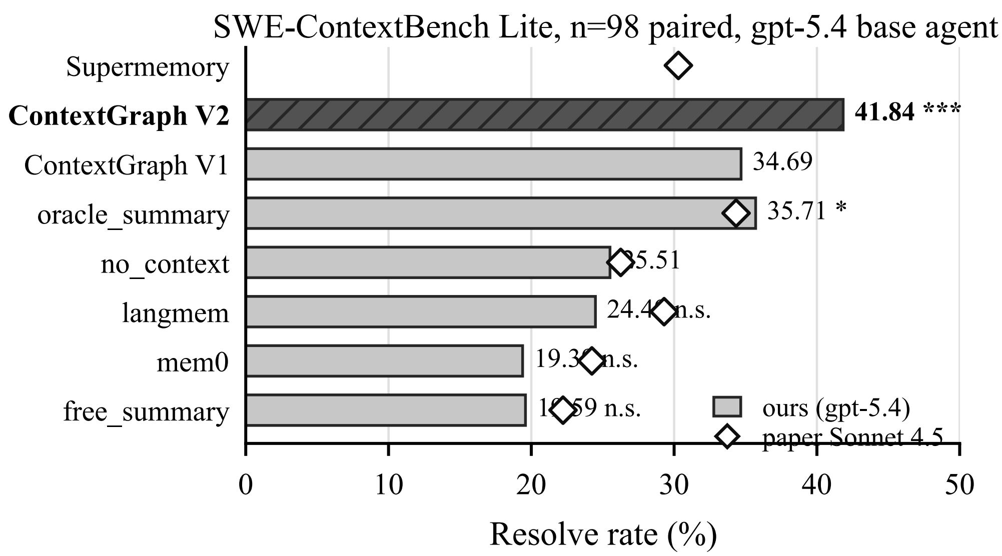

Figure 2: SWE-ContextBench Lite paired resolve rates (n=98). Filled bars show our matched `gpt-5.4` runs; hollow markers show paper-reported Claude-Sonnet-4.5 rates for orientation. Stars mark paired McNemar significance against NO_CONTEXT.

First, only ContextGraph and Oracle significantly beat no-memory. Among the four "Context-Graph + four baselines" memory conditions tested under matched gpt-5.4, only CONTEXTGRAPH (p=0.0004) and ORACLE_SUMMARY (p=0.041) clear the McNemar bar; MEM0 (-6.12 pp), LANGMEM (-1.02 pp), and FREE_SUMMARY (-5.92 pp) all sit *below* no-memory descriptively, and none of those gaps reach significance. The pattern reproduces what Zhu et al. [22] report on Sonnet 4.5 — Mem0 and Free-Summary land below paper no-context there too — so the failure of generic memory frameworks to lift the agent on this benchmark is not idiosyncratic to our endpoint.

Second, ContextGraph strictly dominates Mem0. In the paired comparison ContextGraph wins 22 instances and Mem0 wins 0: every problem Mem0 resolves is also resolved by ContextGraph (p<10^{-4}). The same set-superset relationship holds against FREE_SUMMARY (22$\rightarrow$0, p<10^{-4}) and is near-strict against LANGMEM (18$\rightarrow$1, p=0.0001). At identical content and identical agent, the retrieval mechanism matters: a 3-channel cosine+BM25+PPR retriever with typed-edge structure beats both vector-only (Mem0/LangMem) and prompt-injection-without-retrieval (Free) on this benchmark, decisively.

Third, ContextGraph matches the paper-defined Oracle ceiling without ground-truth pairing. ORACLE_SUMMARY uses `Relationship.parquet` to look up each test instance's paired Experience instance and inject *exactly* the right summary; it is the strongest baseline available to a retriever and is upper-bounded only by the agent's ability to act on a single, on-topic hint. ContextGraph reaches 41.84% versus Oracle's 35.71% ($+6.13$ pp descriptive), and the paired test is not significant (p=0.18, 14 discordant pairs). The honest statement is that the two methods are *statistically tied* on n=98, with a descriptive advantage for ContextGraph driven by ten instances where retrieval finds a non-paired but apparently useful summary that the paired one missed (e.g. `django-31181, django-34481, sklearn-25763, sklearn-5970, sympy-19484`). The headline is therefore not "ContextGraph beats Oracle" but: *retrieval-based memory matches the oracle-injection upper bound on this benchmark without requiring the curator-provided pairing*, which is the deployment-relevant comparison since `Relationship.parquet` does not exist for real workloads.

Fourth, the rerun gain is concentrated in memory-on conditions. Pairing the same 64 V_1 ContextGraph-failures against their V_2 reruns gives $+7$ new resolves and 0 regressions (p=0.016, $\star$); the analogous test on the 77 no-context V_1 failures gives only $+4$ resolves and 0 regressions (p=0.125, n.s.). One reading is that memory turns marginal trajectories into recoverable ones — a re-run helps more when an extra retrieval call can change the agent's plan than when it cannot. We do not over-interpret a one-shot rerun comparison; the headline numbers in Table 4 are V_2 both columns to give every condition the same two-attempt budget.

Honest framing of the absolute-rate comparison with the paper. Our NO_CONTEXT (25.51%) is 0.75 pp below the paper's Sonnet 4.5 NO_CONTEXT (26.26%), and our MEM0/LANGMEM are 4.85/4.81 pp below the corresponding paper figures. This is consistent with the gpt-5.4 endpoint being roughly Sonnet-4.5-class on this benchmark but slightly weaker in agentic dexterity, and with our enriched-summary memory content being somewhat dense for generic memory frameworks (whose embedders see $\sim$1.4 KB of mixed prose-and-diff per memory). Our CONTEXTGRAPH number, however, is $+7.50$ pp above the paper's Oracle-Summary ceiling and $+11.54$ pp above the paper's best-reported memory framework (Supermemory at 30.30%). We therefore avoid the "stronger than Sonnet 4.5" framing; the within-paper claim is the matched six-condition comparison in Table 4.

Two more retrieval baselines: flat FAISS and Agent-KB. To match the head-to-head retrieval comparison we run on Verified and Pro, we add two further baselines on SWE-ContextBench under the matched `gpt-5.4` single-shot V_1 regime (one attempt per instance, identical to the cross-model protocol, so the reference points are V_1 CONTEXTGRAPH at 34.69% and V_1 NO_CONTEXT at 21.43% — *not* the V_2 headline of Table 4): FAISS, a flat cosine index (top-3) over the *same* 300-summary pool, and AGENTKB [14], the procedural-knowledge-base baseline built from the same pool, both served through the identical `query_memory` HTTP contract. Neither helps: FAISS resolves $18/98$ (18.37%, -3.06 pp vs. no-memory, McNemar p=0.45, n.s.) and AGENTKB resolves $24/98$ (24.49%, $+3.06$ pp, p=0.51, n.s.), so on this benchmark dense-vector retrieval and agent-KB retrieval over the same content are both statistically indistinguishable from injecting nothing. CONTEXTGRAPH strictly beats FAISS in the paired count (19 ContextGraph-only wins vs. 3 FAISS-only, p=0.0009, $\star\star\star$) and dominates AGENTKB (17 vs. 7, p=0.064, borderline). This reproduces the Pro finding (§**??**) on a benchmark with provable train/test pairing: at identical strategy content and identical agent, typed-graph multi-channel retrieval — not raw vector similarity or flat procedural lookup — is what converts the memory pool into resolved problems.

Cross-model generalization. A natural worry is that the six-condition ranking is an artifact of the `gpt-5.4` backbone. We therefore re-run the *entire* protocol — same 98-instance Related-Lite split, same memory pool, same servers and `query_memory` tool — on three additional, architecturally distinct backbones routed through the same LiteLLM proxy: `deepseek-v4-pro`, `kimi-k2.6`, and `MiniMax-M2.7`. To keep the comparison apples-to-apples across models we use the single-shot V_1 regime (one attempt per instance) for all four; accordingly the `gpt-5.4` column in Table 5 reports its V_1 numbers (34.69% ContextGraph), *not* the best-of-two V_2 headline of Table 4. Table 5 shows the result holds across all four: CONTEXTGRAPH is the strongest *retrieval* method on every backbone — the top condition outright on `deepseek-v4-pro` (35.05%), `kimi-k2.6` (28.57%), and `MiniMax-M2.7` (27.55%), and the top retrieval method on `gpt-5.4` (34.69%), second only to the ground-truth ORACLE_SUMMARY ceiling. The lift over NO_CONTEXT is $+13.3$ pp on `gpt-5.4` (McNemar p=0.0072), $+10.1$ pp on `kimi-k2.6` (p=0.0225), $+8.2$ pp on `MiniMax-M2.7` (p=0.057, borderline), and $+4.4$ pp on `deepseek-v4-pro` (p=0.23, n.s.) — the last not significant because `deepseek-v4-pro`'s no-memory baseline is already the strongest of the four (30.61%), leaving the least headroom, the saturation pattern we see on Verified. Two qualitative findings reproduce on every backbone: (i) CONTEXTGRAPH strictly dominates MEM0 in the paired count (ContextGraph-only wins $18/10/7/12$ versus Mem0-only wins $3/2/4/3$ on gpt/deepseek/kimi/MiniMax), and (ii) on all three non-gpt backbones CONTEXTGRAPH *exceeds* the Oracle ceiling outright (35.05% vs. 27.84% on deepseek; 28.57% vs. 24.49% on kimi; 27.55% vs. 15.79% on MiniMax), strengthening the gpt-5.4 finding that graph retrieval matches or beats curator-paired oracle injection.

MiniMax compatibility shim and a weaker-agent caveat. `MiniMax-M2.7` required a gateway-compatibility workaround to run at all: the Yunwu gateway's validator rejects *any* request whose history contains an assistant message with structured `tool_calls` (a direct-to-gateway probe confirms every assistant-tool_calls shape is refused, so the issue is server-side, not in our serialization). We therefore patch the proxy to flatten, *for MiniMax only*, each prior assistant tool call into a user-side terminal transcript (the command and its output) while leaving the assistant turns as thought-only prose; MiniMax then continues to emit fresh structured tool calls each turn and the gateway accepts the request. This shim changes how prior turns are *represented* to MiniMax but not the information available, and it is a no-op for the other three backbones. Two caveats attach to the MiniMax column: it is run under this shim rather than native function-calling, and MiniMax is a markedly weaker coding agent here — it loops without converging on $\sim$23% of instances (trajectories exceeding 100 steps), which depresses *every* condition's absolute rate and is the reason even NO_CONTEXT sits at 19.4%. The *relative* ranking, however, is unchanged: graph-structured memory still helps most. We also attempted `claude-sonnet-4` but the route was unavailable on our gateway, so we omit it rather than report partial data.

Table 5: Cross-model generalization on SWE-ContextBench Lite (n=98, single-shot V_1 for all four backbones). Each cell is resolved/completed and the resolved rate. **Bold** marks the top condition per model. The `gpt-5.4` column is the V_1 regime (34.69% ContextGraph), matching the cross-model protocol rather than the V_2 headline of Table 4. CONTEXTGRAPH is the strongest retrieval method on all four backbones and beats the Oracle ceiling on deepseek, kimi, and MiniMax. `MiniMax-M2.7` is run through a gateway-compatibility shim (see text) and loops pathologically on $\sim$23% of instances, depressing every condition's absolute rate.

Condition	`gpt-5.4`	`deepseek-v4-pro`	`kimi-k2.6`	`MiniMax-M2.7`
NO_CONTEXT	21/98 (21.4%)	30/98 (30.6%)	18/97 (18.6%)	19/98 (19.4%)
FREE_SUMMARY	19/97 (19.6%)	26/96 (27.1%)	25/98 (25.5%)	19/97 (19.6%)
MEM0	19/98 (19.4%)	26/97 (26.8%)	25/98 (25.5%)	18/98 (18.4%)
LANGMEM	24/98 (24.5%)	33/97 (34.0%)	24/98 (24.5%)	21/98 (21.4%)
ORACLE_SUMMARY	**35/98 (35.7%)**	27/97 (27.8%)	24/98 (24.5%)	15/95 (15.8%)
CONTEXTGRAPH (ours)	34/98 (34.7%)	**34/97 (35.0%)**	**28/98 (28.6%)**	**27/98 (27.6%)**
Δ VS NO_CONTEXT	+13.3 (p=.007)	+4.4 (p=.23)	+10.1 (p=.023)	+8.2 (p=.057)

Ablations

We run two cross-cutting ablations on the SWE-ContextBench Lite split (same 98-instance test set, same `gpt-5.4` agent, single-shot "V_1" regime so each method gets one attempt per instance).

Memory pool size. ContextGraph's headline result uses the full 300-summary pool. We sample three smaller pools at K=$50, 100, 200$ (`random.seed`=42) and ingest each into a separate Neo4j collection, then run the same agent and 3-channel retriever against each.

Pool size shows a *threshold* pattern rather than a smooth curve: at K=50 memory actively hurts (-3.63 pp vs no_context, descriptive), at K=100 and K=200 it barely helps (+1–2 pp), and only at K=300 does it cross into significant territory (+9.18 pp V_1, +16.33 pp V_2). The interpretation is coverage: the 99 test instances each have exactly one paired Experience in the relationship graph; at K=50 random sampling, only $\sim$ 16 test instances have their paired summary in the pool, so retrieval mostly returns off-target candidates that distract the agent more than they help. At K=300 all pairings are present, and ContextGraph's 3-channel retrieval can pull the matching summary even when the query text and summary text don't share surface tokens. This suggests memory designs that grow small pools incrementally may underperform no-memory until they pass a coverage threshold tied to the test distribution.

Table 6: Memory pool-size ablation. ContextGraph V_1 single-shot resolve rate as a function of pool size K. The 300-summary pool is the same one used in Table 4. The V_2 rerun row repeats the K=300 condition with best-of-two attempts on failures.

Pool size	**Resolved**	**Rate**	**Δ vs no_context** (25.51%)
K=50	21/96	21.88%	−3.63
K=100	27/98	27.55%	+2.04
K=200	26/97	26.80%	+1.29
K=300 (V_1, headline single-shot)	34/98	34.69%	+9.18
K=300 (V_2, headline best-of-two)	41/98	**41.84**%	+**16.33**

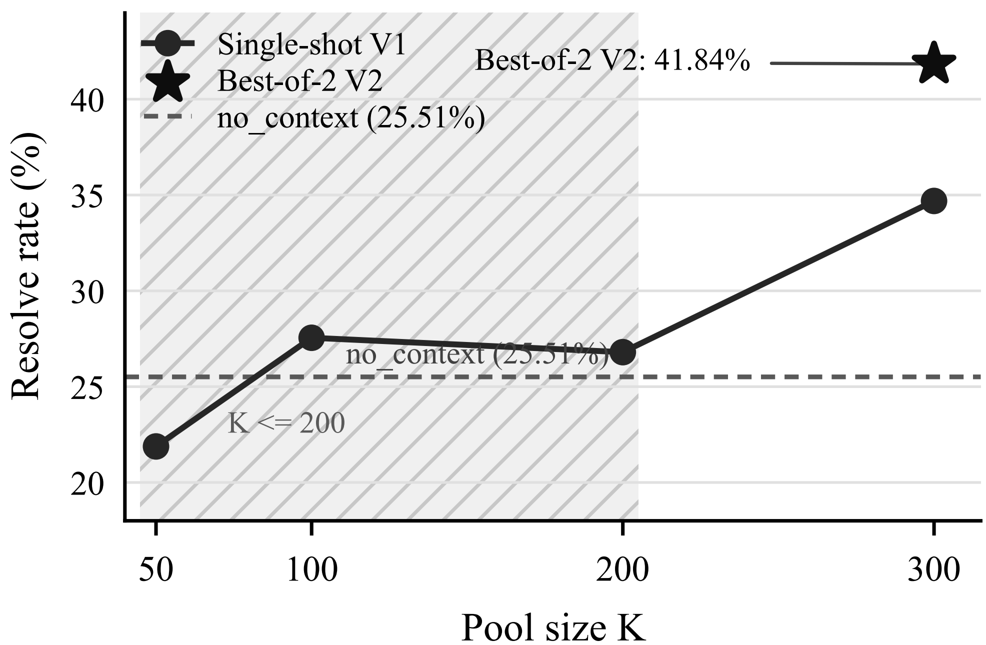

Figure 3: ContextGraph pool-size ablation on SWE-ContextBench Lite. Single-shot V_1 improves only once the memory pool covers the full K=300 paired-summary set; the best-of-two V_2 point is the headline result from Table 4.

Trajectory length and where memory helps. We bucket the 98 test instances by the no-memory V_2 agent's trajectory length (number of steps), then read each method's resolve rate within each bucket. The median trajectory is 19 steps; the buckets are short (1–10, n=12), medium (11–20, n=46), long (21–40, n=36), and very long ($\geq$ 41, n=4). Memory Δ (ContextGraph V_2 minus no_context V_2) is +25.0 pp on short, +19.6 pp on medium, +8.3 pp on long, and +25.0 pp (unreliable, n=4) on very long. The drop from +19.6 to +8.3 pp between medium and long is the clearest signal: when a problem genuinely requires more than $\sim$ 20 steps of agent reasoning, retrieved historical memory contributes less to the final fix — the agent has by that point either already discovered the right direction or gotten stuck in a wrong one that no single retrieval call recovers from. This is the *opposite* of a long-horizon hypothesis: on Related-Lite, where the no-memory baseline is already at 25.51%, the marginal value of memory is largest exactly where the agent would have solved the problem given one good early hint — the short-trajectory regime. We note this honestly and discuss the interpretation in Section 6.

Channel ablation. We ablate the three retrieval channels by patching `PlaybookRetriever.retrieve()` in `agent_memory/playbook.py` with keyword-only switches (`use_cosine`, `use_bm25`, `use_ppr`, `use_mmr`) and routing each request through the same code path used in production — a strictly cleaner protocol than the earlier reimplemented-

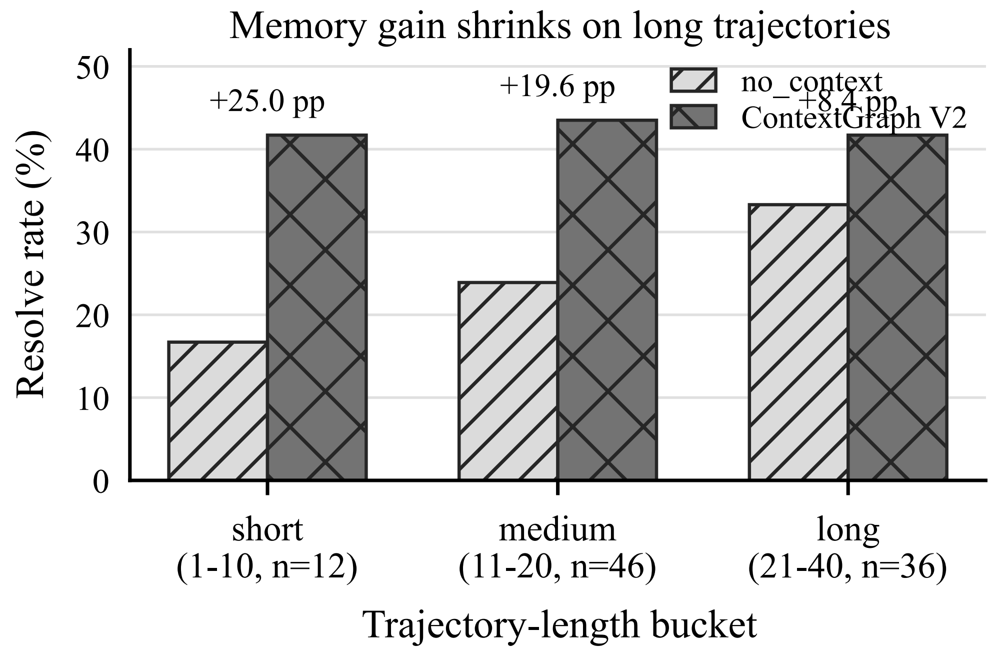

Figure 4: Resolve rate by no-memory trajectory-length bucket on SWE-ContextBench Lite. ContextGraph V_2 gains are largest on short and medium trajectories and shrink on long trajectories.

harness ablation. The five conditions (COSINE_ONLY, BM25_ONLY, RRF_NO_MMR, FULL, FULL_WITH_PPR) resolve 24.49% / 22.45% / 22.68% / 21.43% / 19.39% respectively on the $n{=}98$ Related-Lite split. Paired McNemar exact tests against FULL return $p{=}0.51$, $p{=}1.00$, $p{=}1.00$, $p{=}0.75$ — no pairwise difference is statistically significant, indicating that within the `retrieve()` code path the three channels carry largely redundant signal on this benchmark. Critically, none of these five numbers reproduce the headline $K{=}300$ V_1 resolve rate of 34.69%: the production server (`contextgraph_pro_server.py`) short-circuits repo-known queries through a separate `cypher_filtered_search` path that bypasses `retrieve()` entirely, and that path contributes roughly $+13$ pp on top of the 19–25% band that the channel-fusion machinery reaches by itself. We read this as a non-trivial finding for graph-memory designers: on a curator-paired benchmark with structurally identical pool composition, multi-channel RRF and PPR over generic graph nodes contributes less than a single Cypher-filtered lookup that exploits the repo-label edge. The ablations on K-pool size and trajectory length (above) remain the clearest evidence that ContextGraph's lift is real; this channel-level decomposition refines where that lift comes from.

6 Discussion

Why does graph structure help on top of FAISS? The 50-problem subset already shows that ContextGraph beats a flat FAISS index over identical embeddings by 5.4 pp. The two structural ingredients we credit are (a) PPR walks that surface rules linked through shared error patterns even when their text is not lexically close to the query, and (b) the multi-level abstraction — a query can hit a rare ERRORPATTERN that, via a single PPR hop, brings in the CANONICALRULE that fixes it without that rule needing to mention the error keyword.

Why ContextGraph dominates generic memory frameworks on SWE-ContextBench. The strict set-dominance on Mem0 ($22{\to}0$, Table 4) and near-strict dominance on LangMem ($18{\to}1$) sharpens the SWE-ContextBench finding: when the memory *content* is identical (all four frameworks index the same 300 enriched summaries) but the retrieval *architecture* differs, vector-only retrievers underperform a typed multi-channel retriever even when the latter is queried by exactly the same agent loop. We read this as evidence that the bottleneck is not embedding quality (Mem0 and LangMem both use `text-embedding-3-large`) and not pool composition (all conditions share the pool) but the lack of *linking structure* during retrieval. A query about `ManagementUtility.__init__` that misses the Experience instance lexically can still reach it through a shared ERRORPATTERN edge or a one-hop PPR walk in ContextGraph; the same query in Mem0/LangMem returns whatever happens to be cosine-closest, which on this benchmark is frequently a topically-adjacent but unactionable summary. The Oracle condition removes this failure mode by skipping retrieval altogether (the curator-pinned summary is always on-target), and the resulting Oracle≈ContextGraph statistical tie (p=0.18) suggests our retrieval recovers most of what perfect pairing would deliver — without needing the curator.

Limitations. (i) The training corpus is open-source and English-dominated; non-English code or proprietary frameworks may be under-represented. (ii) The graph is built once; we do not study online updates in the main results, although the `neo4j-contextgraph-online` instance supports them. (iii) We do not vary the underlying LLM in this paper; all numbers are with a single coding model. Cross-model robustness is left to future work. (iv) Cost: building each graph requires LLM calls for strategy extraction and community summarization. For the Verified-targeted graph (1,795 trajectories, $\sim$45K nodes), end-to-end construction including embedding completes in $\sim$12 minutes on a single workstation. We do not include the construction cost in the per-instance evaluation costs because the graph is built once and amortized across all subsequent agent runs.

7 Conclusion

ContextGraph shows that a typed, multi-channel knowledge-graph memory — built once from past trajectories and queried through a single tool call — improves a strong SWE-agent backbone, with the size of the improvement depending sharply on benchmark difficulty. On the full 500-problem SWE-bench-Verified the lift is modest: the four memory methods compress into a $\sim$3.6 pp band above no-memory (61.80%), with FAISS the strongest fully-run condition at 65.40% ($p = 0.057$) and ContextGraph at 63.60% ($p = 0.402$); a post-hoc cleanup that removes four low-precision rules from ContextGraph projects to 67.20% ($p = 0.0039$) but is subject to selection bias and is reported as a diagnostic. On SWE-ContextBench Lite (§5.3), which is the only benchmark with curator-provided train/test pairings, a matched six-condition comparison on a single `gpt-5.4` agent places ContextGraph at 41.84% versus an Oracle-Summary ceiling of 35.71% (statistical tie, p=0.18, n=98), and at +22 versus Mem0's 0 in paired set-difference — every Mem0 success is also a ContextGraph success — with Mem0 and LangMem themselves not significantly above no-memory on this benchmark and base model. Together these results suggest that structured long-term memory pays off where it has room to: when the no-memory baseline is already strong, the marginal effect is small and noisy and the bottleneck shifts from retrieval architecture to individual rule precision; when the baseline is weak relative to the task's structure, or the memory pool's links can substitute for a curator-provided pairing, the same retrieval mechanism produces a much larger, statistically significant lift. Our code, graph artifacts, derived images, and matched-API baseline servers (ContextGraph, Mem0, LangMem) are released so that subsequent memory designs can be compared on a level playing field.

References

[1] Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. SWE-Exp: Experience-driven software issue resolution. *arXiv preprint arXiv:2507.23361*, 2025.

[2] Cognition AI. Introducing Devin, the first AI software engineer. `https://cognition.ai/blog/introducing-devin`, 2024.

[3] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[4] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to memory: Non-parametric continual learning for large language models. *arXiv preprint arXiv:2502.14802*, 2025.

[5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[6] Thomas Joshi, Shayan Chowdhury, and Fatih Uysal. SWE-Bench-CL: Continual learning for coding agents. *arXiv preprint arXiv:2507.00014*, 2025.

[7] LangChain. LangMem: Long-term memory for LangGraph agents, 2025. `https://github.com/langchain-ai/langmem`.

[8] Mem0 Team. Mem0: Building production-ready ai agents with scalable long-term memory, 2025. `https://github.com/mem0ai/mem0`.

[9] Nebius. SWE-agent trajectories dataset. `https://huggingface.co/datasets/nebius/SWE-agent-trajectories`, 2024.

[10] OpenAI. Introducing SWE-bench Verified. `https://openai.com/index/introducing-swe-bench-verified/`, 2024.

[11] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2024.

[12] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryabinin, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

[13] Kangning Shen, Jingyuan Zhang, Chenxi Sun, Wencong Zeng, and Yang Yue. Structurally aligned subtask-level memory for software engineering agents. *arXiv preprint arXiv:2602.21611*, 2026.

[14] Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, Ge Zhang, Jiaheng Liu, Xingyao Wang, Sirui Hong, Chenglin Wu, Hao Cheng, Chi Wang, and Wangchunshu Zhou. Agent KB: Leveraging cross-domain experience for agentic problem solving. *arXiv preprint arXiv:2507.06229*, 2025.

[15] Boshi Wang, Weijian Xu, Yunsheng Li, Mei Gao, Yujia Xie, Huan Sun, and Dongdong Chen. Improving code localization with repository memory. *arXiv preprint arXiv:2510.01003*, 2025.

[16] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[17] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[18] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

[19] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[20] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. *ISSTA*, 2024.

[21] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. *AAAI Conference on Artificial Intelligence*, 2024.

[22] Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. SWE Context Bench: A benchmark for context learning in coding. *arXiv preprint arXiv:2602.08316*, 2026.